Exp 8: Implement Multiclass Classification models for students Performance Level analysis and perform comparative analysis. (Random Forest, Decision Tree, Multinomial Logistic Regression, XGBoost, K-Nearest Neighbors).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder,StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,classif
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/multiclass_classification_dataset_500_samples.csv')
df.head()
```

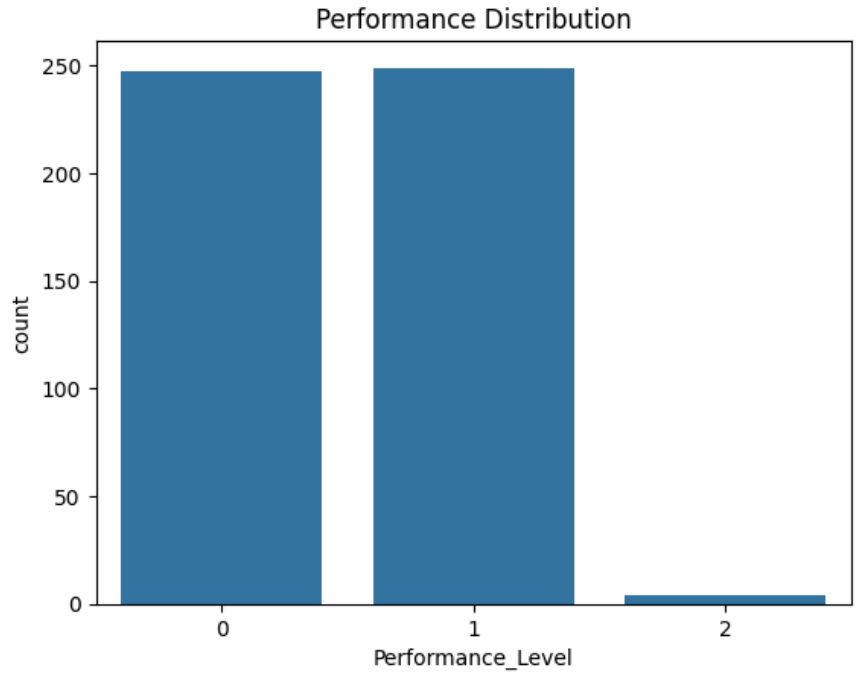
	Study_Hours	Attendance_Percentage	Assignment_Score	Internal_Marks	Participation	Internet_Access	Pr
0	22.48	84.26	86.79	76.68	Medium	Yes	
1	19.31	94.09	81.10	56.73	Low	Yes	
2	23.24	61.01	70.72	52.73	Medium	Yes	
3	27.62	80.63	62.24	64.95	Low	Yes	
4	18.83	68.49	78.38	62.45	High	Yes	

Next steps: Generate code with df New interactive sheet

```
print(df.shape)
df.describe()
df.info()
```

```
(500, 8)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Study_Hours            500 non-null    float64
 1   Attendance_Percentage  500 non-null    float64
 2   Assignment_Score       500 non-null    float64
 3   Internal_Marks         500 non-null    float64
 4   Participation          500 non-null    object
 5   Internet_Access        500 non-null    object
 6   Previous_Grade         500 non-null    object
 7   Performance_Level      500 non-null    int64
dtypes: float64(4), int64(1), object(3)
memory usage: 31.4+ KB
```

```
sns.countplot(x="Performance_Level",data=df)
plt.title("Performance Distribution")
plt.show()
```




```
df=pd.get_dummies(df,drop_first=True)
df.head()
```

	Study_Hours	Attendance_Percentage	Assignment_Score	Internal_Marks	Performance_Level	Participation_I
0	22.48	84.26	86.79	76.68	1	Fa
1	19.31	94.09	81.10	56.73	1	T
2	23.24	61.01	70.72	52.73	0	Fa
3	27.62	80.63	62.24	64.95	0	T
4	18.83	68.49	78.38	62.45	0	Fa

Next steps: Generate code with df New interactive sheet

```
X=df.drop("Performance_Level",axis=1)
y=df["Performance_Level"]
```

```
sc=StandardScaler()
X=sc.fit_transform(X)
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)
```

```
models={
    "Random Forest":RandomForestClassifier(),
    "Decision Tree":DecisionTreeClassifier(),
    "Logistic Regression":LogisticRegression(max_iter=2000),
    "KNN":KNeighborsClassifier(),
    "XGBoost":XGBClassifier(eval_metric="mlogloss")
}
```

```
results=[]
for name,model in models.items():
    model.fit(X_train,y_train)
    pred=model.predict(X_test)
    acc=accuracy_score(y_test,pred)
    pre=precision_score(y_test,pred,average="weighted")
    rec=recall_score(y_test,pred,average="weighted")
    f1w=f1_score(y_test,pred,average="weighted")
    f1m=f1_score(y_test,pred,average="macro")
    f1mi=f1_score(y_test,pred,average="micro")
```

```
    f1mi=f1_score(y_test,pred,average="micro")
    results.append([name,acc,pre,rec,f1w,f1m,f1mi])
results_df=pd.DataFrame(results,columns=[
    "Model","Accuracy","Precision","Recall",
    "F1_weighted","F1_macro","F1_micro"])
results_df
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

1 to 5 of 5 entries Filter

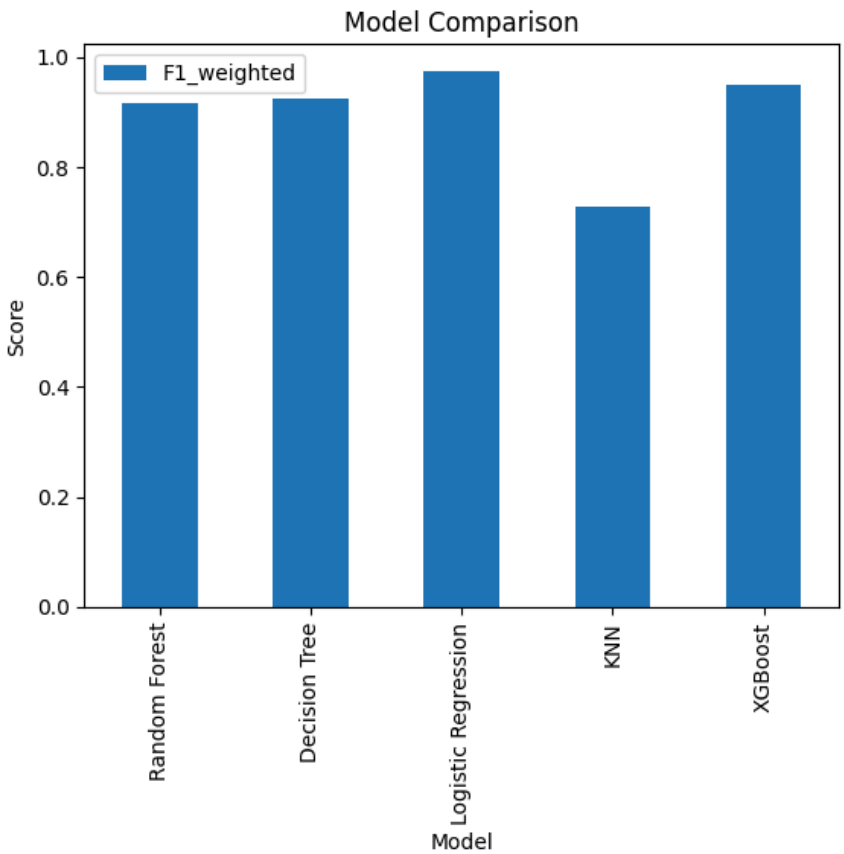
index	Model	Accuracy	Precision	Recall	F1_weighted	F1_macro	F1_micro
0	Random Forest	0.93	0.9263665466186474	0.93	0.9253210077204389	0.6231206826493295	0.93
1	Decision Tree	0.93	0.9231232436772382	0.93	0.9253700305810397	0.6230377166156983	0.93
2	Logistic Regression	0.98	0.9702272727272728	0.98	0.9750574712643677	0.6568828680897646	0.98
3	KNN	0.73	0.736728221597752	0.73	0.7267455501618123	0.48901024811218985	0.73
4	XGBoost	0.95	0.9520426579163248	0.95	0.9501546232347181	0.9659828883620246	0.95

Show 25 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.

Next steps: Generate code with `results_df` New interactive sheet

```
results_df.plot(x="Model",y="F1_weighted",kind="bar")
plt.title("Model Comparison")
plt.ylabel("Score")
plt.show()
```

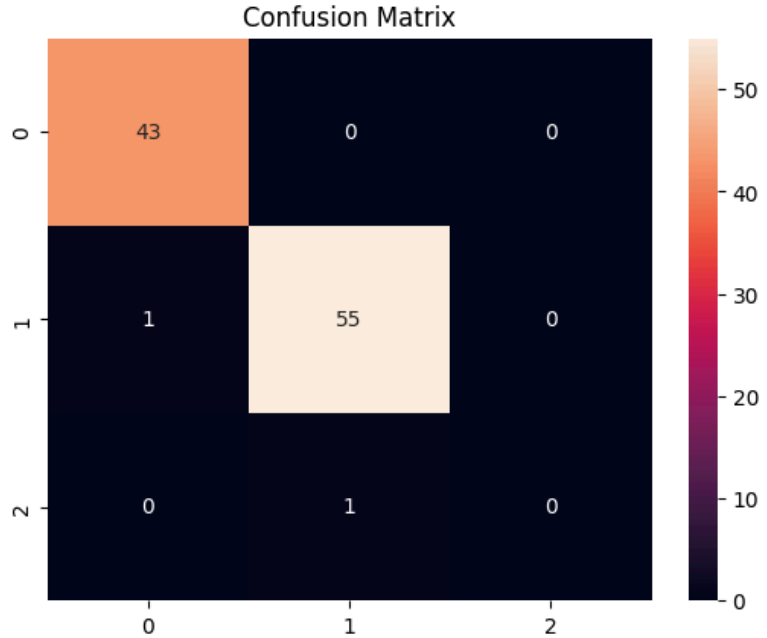



```
best_name=results_df.sort_values("F1_weighted",ascending=False).iloc[0,0]
best_model=models[best_name]
print("Best Model:",best_name)
```

```
Best Model: Logistic Regression
```

```
with(open("best_model","wb")) as f:
  pickle.dump(best_model,f)
```

```
pred=best_model.predict(X_test)
cm=confusion_matrix(y_test,pred)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("Confusion Matrix")
plt.show()
```




```
print(classification_report(y_test,pred))
```

```
              precision    recall  f1-score   support

           0       0.98      1.00      0.99        43
           1       0.98      0.98      0.98        56
           2       0.00      0.00      0.00         1

    accuracy                           0.98       100
   macro avg       0.65      0.66      0.66       100
weighted avg       0.97      0.98      0.98       100

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```